



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

«МИРЭА – Российский технологический университет»

**РТУ МИРЭА**

Институт искусственного интеллекта

Кафедра проблем управления

**ПРАКТИЧЕСКАЯ РАБОТА №5**

по дисциплине: **Информационные элементы робототехнических систем**

**Тема лабораторной работы:** «Обучение детектора на изображениях для нахождения фруктов на картинках»

**Студенты группы:** КРБО-03-  
23

Зенина А.А. \_\_\_\_\_

Грачев А.В. \_\_\_\_\_

Гришаев А.К. \_\_\_\_\_

**Преподаватель:**

ст. преподаватель Бычков А.М. \_\_\_\_\_

Работа представлена к  
защите:

«\_\_\_» \_\_\_\_\_ 2026 г.

Москва 2026

## Импорт библиотек и переход на cuda

```
import subprocess
import sys

def install_package(package):
    subprocess.check_call([sys.executable, "-m", "pip", "install", package])

packages = ['torch', 'torchvision', 'matplotlib', 'pillow', 'opencv-python',
            'numpy', 'pandas', 'tqdm', 'seaborn', 'kagglehub']

for package in packages:
    try:
        __import__(package)
    except ImportError:
        print(f"Установка {package}...")
        install_package(package)

import torch
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
import torchvision.transforms as T
from torch.utils.data import DataLoader, Dataset
import torch.optim as optim

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from PIL import Image
import cv2
import os
import random
from pathlib import Path
from tqdm.notebook import tqdm
import json
import requests
from io import BytesIO
import zipfile
import shutil

torch.set_float32_matmul_precision('high') # Использовать TensorFloat32
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Используется устройство: {device}")
```

```
if torch.cuda.is_available():  
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

Установка pillow...

Установка opencv-python...

Используется устройство: cuda

GPU: NVIDIA GeForce RTX 4060 Laptop GPU

**Загрузка датасета с помощью Kaggle** Загружается датасет moltean/fruits через kagglehub.

**Производится поиск подпапки fruits-360\_100x100** внутри загруженных данных.

**Определяются пути к тренировочной и тестовой папкам.**

**Извлекаются и выводятся названия всех 257 классов фруктов/овощей.**

**В случае ошибки (например, отсутствие доступа к Kaggle) создаётся демонстрационный датасет из случайных цветных квадратов для тестирования пайплайна.**

```
print("Настройка Kaggle API...")
```

```
if not os.path.exists(os.path.expanduser('~/.kaggle')):  
    os.makedirs(os.path.expanduser('~/.kaggle'))
```

```
dataset_name = "moltean/fruits"
```

```
print(f"Загрузка датасета {dataset_name}...")
```

```
try:
```

```
    import kagglehub
```

```
    path = kagglehub.dataset_download(dataset_name)
```

```
    print(f"Датасет загружен в: {path}")
```

```
    # Ищем папку fruits-360_100x100
```

```
    dataset_path = None
```

```
    for root, dirs, files in os.walk(path):
```

```
        if "fruits-360_100x100" in root:
```

```
            dataset_path = root
```

```
            break
```

```
    if dataset_path:
```

```
        print(f"\nНайден датасет fruits-360_100x100: {dataset_path}")
```

```

# Структура: fruits-360_100x100/fruits-360/Training/ u Test/
train_path = os.path.join(dataset_path, "fruits-360", "Training")
test_path = os.path.join(dataset_path, "fruits-360", "Test")

print(f"Тренировочная папка: {train_path}")
print(f"Тестовая папка: {test_path}")

classes = []
if os.path.exists(train_path):
    for folder in sorted(os.listdir(train_path)):
        folder_path = os.path.join(train_path, folder)
        if os.path.isdir(folder_path):
            classes.append(folder)

    print(f"\nВСЕГО классов в датасете: {len(classes)}")
    print(f"Первые 20 классов: {classes[:20]}")
    print(f"Последние 20 классов: {classes[-20:]}")

else:
    raise Exception("Папка fruits-360_100x100 не найдена")

except Exception as e:
    print(f"Ошибка: {e}")
    print("\nСоздаем демонстрационный датасет...")

os.makedirs("demo_dataset/Training", exist_ok=True)
os.makedirs("demo_dataset/Test", exist_ok=True)

from PIL import ImageDraw

# Для демо берем 10 классов
classes = ['Apple', 'Banana', 'Orange', 'Grape', 'Strawberry',
           'Pear', 'Peach', 'Plum', 'Cherry', 'Kiwi']

for class_name in classes:
    os.makedirs(f"demo_dataset/Training/{class_name}", exist_ok=True)
    os.makedirs(f"demo_dataset/Test/{class_name}", exist_ok=True)

    for i in range(20):
        img = Image.new('RGB', (100, 100), color=(random.randint(0,255),
                                                    random.randint(0,255),
                                                    random.randint(0,255)))
        img.save(f"demo_dataset/Training/{class_name}/{i}.jpg")
        if i < 10:
            img.save(f"demo_dataset/Test/{class_name}/{i}.jpg")

train_path = "demo_dataset/Training"

```

```
val_path = "demo_dataset/Test"
print(f"\nДемонстрационный датасет создан в: demo_dataset")
```

Настройка Kaggle API...

Загрузка датасета moltean/fruits...

Датасет загружен в:

C:\Users\admin\_local\.cache\kagglehub\datasets\moltean\fruits\versions\87

Найден датасет fruits-360\_100x100:

C:\Users\admin\_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360\_100x100

Тренировочная папка:

C:\Users\admin\_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360\_100x100\fruits-360\Training

Тестовая папка:

C:\Users\admin\_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360\_100x100\fruits-360\Test

ВСЕГО классов в датасете: 257

Первые 20 классов: ['Almonds 1', 'Apple 10', 'Apple 11', 'Apple 12', 'Apple 13', 'Apple 14', 'Apple 17', 'Apple 18', 'Apple 19', 'Apple 20', 'Apple 21', 'Apple 22', 'Apple 23', 'Apple 5', 'Apple 6', 'Apple 7', 'Apple 8', 'Apple 9', 'Apple Braeburn 1', 'Apple Crimson Snow 1']

Последние 20 классов: ['Tomato 3', 'Tomato 4', 'Tomato 5', 'Tomato 7', 'Tomato 8', 'Tomato 9', 'Tomato Cherry Maroon 1', 'Tomato Cherry Orange 1', 'Tomato Cherry Red 1', 'Tomato Cherry Red 2', 'Tomato Cherry Yellow 1', 'Tomato Heart 1', 'Tomato Maroon 1', 'Tomato Maroon 2', 'Tomato Yellow 1', 'Walnut 1', 'Watermelon 1', 'Zucchini 1', 'Zucchini Green 1', 'Zucchini dark 1']

**Подготовка данных** Определяются переменные с путями и списком классов из предыдущей ячейки (если они не определены, используются запасные демо-пути).

**Выводится информация о путях и количестве классов для контроля.**

**try:**

```
train_path
val_path
classes
```

**except NameError:**

```
print("Переменные не найдены. Используем стандартные пути...")
train_path = "demo_dataset/Training"
val_path = "demo_dataset/Test"
classes = ['Apple', 'Banana', 'Orange', 'Grape', 'Strawberry']
```

```
print(f"\nДатасет настроен:")
print(f" Тренировочный путь: {train_path}")
print(f" Валидационный путь: {val_path}")
print(f" Количество классов: {len(classes)}")
```

```
print(f" Первые 5 классов: {classes[:5]}")
print(f" Последние 5 классов: {classes[-5:]}")
```

Датасет настроен:

Тренировочный путь:

C:\Users\admin\_local\.cache\kagglehub\datasets\moltean\fruits\versions\87\fruits-360\_100x100\fruits-360\Training

Валидационный путь: demo\_dataset/Test

Количество классов: 257

Первые 5 классов: ['Almonds 1', 'Apple 10', 'Apple 11', 'Apple 12', 'Apple 13']

Последние 5 классов: ['Walnut 1', 'Watermelon 1', 'Zucchini 1', 'Zucchini Green 1', 'Zucchini dark 1']

**Создание датасета с выбранными классами** Определяется класс **FruitsDetectionDataset**, который:

**Загружает изображения только указанных классов (по умолчанию 6 фруктов: Apple 1, Banana 1, Orange 1, Strawberry 1, Grape 1, Pear 1).**

**Реализует аугментацию для тренировочной выборки: удаление белого фона, замена на случайный фон (сплошной, градиент или шум), масштабирование, сдвиг, изменение размера bounding box, поворот.**

**Применяет нормализацию и преобразование в тензоры.**

**Выбираются доступные классы из общего списка (поиск по ключевым словам).**

**Создаются объекты train\_dataset и val\_dataset с ограничением количества изображений на класс (100 для тренировки, 50 для валидации).**

**Выводится статистика: размер обучающей и валидационной выборок, количество классов.**

```
import random
import numpy as np
import cv2
from PIL import Image

class FruitsDetectionDataset(Dataset):
    """Датасет для детекции фруктов с аугментацией"""
    def __init__(self, root_dir, class_list, transforms=None,
samples_per_class=None, is_training=True):
        self.root_dir = root_dir
        self.class_list = class_list
        self.transforms = transforms
        self.is_training = is_training
        self.class_to_idx = {cls: idx+1 for idx, cls in
```

```

enumerate(class_list)}
    self.idx_to_class = {idx+1: cls for idx, cls in
enumerate(class_list)}

    self.images = []
    self.labels = []

    for class_name in class_list:
        class_path = os.path.join(root_dir, class_name)
        if os.path.exists(class_path):
            images_in_class = [f for f in os.listdir(class_path)
                               if f.lower().endswith(('.jpg', '.jpeg',
'.png'))]

            if samples_per_class:
                selected = images_in_class[:samples_per_class]
            else:
                selected = images_in_class

            for img_name in selected:
                self.images.append(os.path.join(class_path, img_name))
                self.labels.append(class_name)

    print(f" Загружено {len(self.images)} изображений,
{len(set(self.labels))} классов")

    def remove_white_background(self, image):

        img_np = np.array(image)

        if img_np.shape[-1] == 4:
            img_np = img_np[:, :, :3]

        white_mask = np.all(img_np >= 240, axis=2)
        object_mask = ~white_mask
        object_mask = object_mask.astype(np.uint8) * 255

        kernel = np.ones((3, 3), np.uint8)
        object_mask = cv2.morphologyEx(object_mask, cv2.MORPH_OPEN, kernel)
        object_mask = cv2.morphologyEx(object_mask, cv2.MORPH_CLOSE, kernel)

        contours, _ = cv2.findContours(object_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        if contours:
            largest = max(contours, key=cv2.contourArea)
            clean_mask = np.zeros_like(object_mask)
            cv2.drawContours(clean_mask, [largest], -1, 255, -1)
            object_mask = clean_mask

```

```

        return object_mask > 0

    def add_random_background(self, image, boxes):
        """Заменяет белый фон на случайный цвет"""
        if self.is_training and random.random() < 0.4:
            img_np = np.array(image)
            h, w = img_np.shape[:2]

            object_mask = self.remove_white_background(image)

            if np.any(object_mask):
                bg_type = random.choice(['solid', 'gradient', 'noise'])

                if bg_type == 'solid':
                    bg_color = (random.randint(50, 200),
                                random.randint(50, 200),
                                random.randint(50, 200))
                    new_bg = np.full((h, w, 3), bg_color, dtype=np.uint8)

                elif bg_type == 'gradient':
                    new_bg = np.zeros((h, w, 3), dtype=np.uint8)
                    color1 = (random.randint(50, 150), random.randint(50,
150), random.randint(50, 150))
                    color2 = (random.randint(150, 250), random.randint(150,
250), random.randint(150, 250))
                    for i in range(w):
                        ratio = i / w
                        color = tuple(int(c1 * (1-ratio) + c2 * ratio) for
c1, c2 in zip(color1, color2))
                        new_bg[:, i] = color

                else:
                    new_bg = np.random.randint(100, 200, (h, w, 3),
dtype=np.uint8)

            result = new_bg.copy()
            result[object_mask]
            = img_np[object_mask]

            return Image.fromarray(result), boxes

    return image, boxes

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_path = self.images[idx]

        try:

```



```

        image = Image.open(img_path).convert("RGB")
    except Exception as e:
        print(f"Ошибка загрузки {img_path}: {e}")
        image = Image.new('RGB', (100, 100), color=(128, 128, 128))

    width, height = image.size
    original_width, original_height = width, height

    content_margin = int(min(original_width, original_height) * 0.15)
    boxes = [[content_margin, content_margin,
                original_width - content_margin, original_height -
content_margin]]

    if self.is_training:

        image, boxes = self.add_random_background(image, boxes)

    if random.random() < 0.5:
        scale = random.uniform(0.85, 1.15)
        new_w = int(original_width * scale)
        new_h = int(original_height * scale)
        image = image.resize((new_w, new_h))
        boxes = [[int(b[0] * scale), int(b[1] * scale),
                    int(b[2] * scale), int(b[3] * scale)] for b in
boxes]

        width, height = new_w, new_h

    if random.random() < 0.5:
        max_shift_x = int(width * 0.1)
        max_shift_y = int(height * 0.1)
        shift_x = random.randint(-max_shift_x, max_shift_x)
        shift_y = random.randint(-max_shift_y, max_shift_y)
        boxes = [[b[0] + shift_x, b[1] + shift_y,
                    b[2] + shift_x, b[3] + shift_y] for b in boxes]

    if random.random() < 0.5:
        noise = random.uniform(0.90, 1.10)
        x1 = int(boxes[0][0] * noise)
        y1 = int(boxes[0][1] * noise)
        x2 = int(boxes[0][2] * (2 - noise))
        y2 = int(boxes[0][3] * (2 - noise))
        x1 = max(0, min(x1, width//2))
        y1 = max(0, min(y1, height//2))
        x2 = min(width, max(x2, width//2))
        y2 = min(height, max(y2, height//2))

```

```

        boxes = [[x1, y1, x2, y2]]

        if random.random() < 0.3:
            angle = random.uniform(-15, 15)
            image = image.rotate(angle, expand=False)

        if self.transforms:
            image = self.transforms(image)
        else:
            image = T.ToTensor()(image)

        boxes = [[max(0, min(b[0], width)), max(0, min(b[1], height)),
                  max(0, min(b[2], width)), max(0, min(b[3], height))] for b
in boxes]
        boxes = torch.as_tensor(boxes, dtype=torch.float32)

        class_name = self.labels[idx]
        label = self.class_to_idx.get(class_name, 1)
        labels = torch.as_tensor([label], dtype=torch.int64)

        target = {
            'boxes': boxes,
            'labels': labels,
            'image_id': torch.tensor([idx]),
            'area': (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:,
1])),
            'iscrowd': torch.zeros((1,), dtype=torch.int64)
        }

        return image, target

```

```

SELECTED_CLASSES = [
    'Apple 1',      # Яблоко
    'Banana 1',     # Банан
    'Orange 1',     # Апельсин
    'Strawberry 1', # Клубника
    'Grape 1',      # Виноград
    'Pear 1'        # Груша
]

def get_available_classes(full_class_list, selected_classes):
    """
    Проверяет наличие выбранных классов в датасете
    """
    available = []

```

```

for cls in selected_classes:

    found = None
    for c in full_class_list:
        if cls == c or cls in c or c in cls:
            found = c
            break
    if found:
        available.append(found)
        print(f"    Найден класс: {found}")
    else:
        print(f"    Не найден класс: {cls}")

return available

# Трансформации
transform =
    T.Compose([ T.Resize((100,
100)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

print("\n" + "="*50)
print("ПОИСК КЛАССОВ В ДАТАСЕТЕ")
print("="*50)

used_classes = get_available_classes(classes, SELECTED_CLASSES)

if len(used_classes) < 6:
    print(f"\n Найдено только {len(used_classes)} классов, ищем
дополнительные...")
    fruit_keywords = ['Apple', 'Banana', 'Orange', 'Strawberry', 'Grape',
'Pear', 'Peach', 'Plum', 'Cherry', 'Kiwi', 'Mango']

    for cls in classes:
        for keyword in fruit_keywords:
            if keyword in cls and cls not in used_classes:
                used_classes.append(cls)
                print(f"    Добавлен: {cls}")
                break
        if len(used_classes) >= 6:
            break

used_classes = used_classes[:6]

print(f"\n{'='*50}")

```

```

print(f" ВЫБРАННЫЕ КЛАССЫ (всего {len(used_classes)}):") for i,
cls in enumerate(used_classes, 1):
    print(f"    {i}. {cls}")
print(f"{'='*50}")

print("\n" + "="*50)
print("СОЗДАНИЕ ДАТАСЕТА (100 ИЗОБРАЖЕНИЙ НА КЛАСС)")
print("="*50)

```

SAMPLES\_PER\_CLASS = 100 # 100 изображений на класс

```

train_dataset =
    FruitsDetectionDataset( root_dir=train_path,
        class_list=used_classes,
        transforms=transform,
        samples_per_class=SAMPLES_PER_CLASS,
        is_training=True
    )

```

```

val_dataset =
    FruitsDetectionDataset( root_dir=val_path,
        class_list=used_classes,
        transforms=transform,
        samples_per_class=SAMPLES_PER_CLASS // 2,
        is_training=False
    )

```

```

print(f"\n Статистика датасета:")
print(f"    Всего классов: {len(used_classes)}")
print(f"    Изображений на класс в TRAIN: {SAMPLES_PER_CLASS}")
print(f"    Изображений на класс в VAL: {SAMPLES_PER_CLASS // 2}")

```

```

print(f"\n Размер обучающей выборки: {len(train_dataset)}")
print(f" Размер валидационной выборки: {len(val_dataset)}")
print(f" Количество классов: {len(used_classes)}")

```

```

=====
ПОИСК КЛАССОВ В ДАТАСЕТЕ
=====

```

```

    Найден класс: Apple 10
    Найден класс: Banana 1
    Найден класс: Orange 1
    Найден класс: Strawberry 1
    Найден класс: Grape 1
    Найден класс: Pear 1

```

```
=====
ВЫБРАННЫЕ КЛАССЫ (всего 6):
1. Apple 10
2. Banana 1
3. Orange 1
4. Strawberry 1
5. Grape 1
6. Pear 1
=====
```

```
=====
СОЗДАНИЕ ДАТАСЕТА (100 ИЗОБРАЖЕНИЙ НА КЛАСС)
=====
Загружено 600 изображений, 6 классов
Загружено 0 изображений, 0 классов
```

```
Статистика датасета: Всего
классов: 6
Изображений на класс в TRAIN: 100
Изображений на класс в VAL: 50
```

```
Размер обучающей выборки: 600
Размер валидационной выборки: 0
Количество классов: 6
```

**Создание модели** Создаётся модель `fasterrcnn_resnet50_fpn` с предобученными весами (COCO).

**Заменяется классификационная головка (box predictor) на новую, соответствующую количеству классов (фрукты + фон).**

**Модель перемещается на вычислительное устройство (GPU/CPU).**

**Выводится количество всех и обучаемых параметров модели.**

```
def create_model(num_classes):
    """Создание модели Faster R-CNN"""
    model = fasterrcnn_resnet50_fpn(weights='DEFAULT')
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)
    return model

num_classes = len(used_classes) + 1
print(f"Количество классов (включая фон): {num_classes}")
```

```

model = create_model(num_classes)
model.to(device)

print(f"Модель создана и перемещена на {device}")

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print(f" Всего параметров: {total_params:,}") print(f"
Обучаемых параметров: {trainable_params:,}")

```

Количество классов (включая фон): 7  
 Модель создана и перемещена на cuda  
 Всего параметров: 41,324,786  
 Обучаемых параметров: 41,102,386

**DataLoader и обучение (с диагностикой GPU)** Определяется функция `collate_fn` для упаковки батчей (так как изображения и цели могут иметь разный размер).

**Проверяется наличие данных и GPU (объём видеопамяти до обучения).**

**Создаются DataLoader для тренировки и валидации (валидационный датасет может быть пустым).**

**При необходимости модель создаётся повторно (если переменная `model` не определена).**

**Настраивается оптимизатор SGD с `momentum` и `weight decay`.**

**Реализована функция `train_one_epoch`, которая:**

**Переносит изображения и цели на GPU.**

**Вычисляет `loss`, выполняет `backpropagation` и шаг оптимизатора.**

**Отслеживает использование VRAM.**

**Запускается обучение на 10 эпох, сохраняются значения функции потерь. По окончании строится график уменьшения `loss`.**

```

def collate_fn(batch):
    return tuple(zip(*batch))

# Проверка перед созданием DataLoader
print(f"\nПроверка перед созданием DataLoader:")
print(f" Train dataset size: {len(train_dataset)}")
print(f" Val dataset size: {len(val_dataset)}")
print(f" GPU доступен: {torch.cuda.is_available()}")

```

```

print(f" Устройство: {device}")
if torch.cuda.is_available():
    print(f" GPU: {torch.cuda.get_device_name(0)}")
    print(f" VRAM всего: {torch.cuda.get_device_properties(0).total_memory /
1024**3:.1f} GB")
    print(f" VRAM занято до обучения: {torch.cuda.memory_allocated(0) /
1024**3:.2f} GB")

if len(train_dataset) == 0:
    raise ValueError("Невозможно создать DataLoader: обучающий датасет
пуст!")

train_loader =
    DataLoader( train_dataset,
        batch_size=min(4, len(train_dataset)),
        shuffle=True,
        num_workers=0,
        collate_fn=collate_fn
    )

if len(val_dataset) > 0:
    val_loader = DataLoader(
        val_dataset,
        batch_size=min(2, len(val_dataset)),
        shuffle=False,
        num_workers=0,
        collate_fn=collate_fn
    )
    print(f"\nDataLoader созданы успешно!")
    print(f" Количество батчей в train: {len(train_loader)}")
    print(f" Количество батчей в val: {len(val_loader)}")
else:
    val_loader = None
    print(f"\n ВНИМАНИЕ: Валидационный датасет пуст!") print(f"
DataLoader для обучения создан:")
    print(f" Количество батчей в train: {len(train_loader)}")

if 'model' not in dir():
    print("\nСоздаем модель...")

    def create_model(num_classes):
        model = fasterrcnn_resnet50_fpn(weights='DEFAULT')
        in_features = model.roi_heads.box_predictor.cls_score.in_features
        model.roi_heads.box_predictor = FastRCNNPredictor(in_features,
num_classes)
        return model

    num_classes = len(used_classes) + 1

```

```

model = create_model(num_classes)
model.to(device)
print(f"Модель создана на {device}")

print(f"\n Модель на устройстве: {next(model.parameters()).device}")

optimizer = optim.SGD(
    [p for p in model.parameters() if p.requires_grad],
    lr=0.005,
    momentum=0.9,
    weight_decay=0.0005
)

def train_one_epoch(model, optimizer, data_loader, device, epoch):

    model.train()
    total_loss = 0
    num_batches = 0

    if len(data_loader) == 0:
        print(f"Предупреждение: data_loader пуст на эпохе {epoch}")
        return 0

    progress_bar = tqdm(data_loader, desc=f"Epoch {epoch}")

    for batch_idx, (images, targets) in enumerate(progress_bar):
        try:

            if batch_idx == 0:
                print(f"\n Тензоры ДО перемещения на GPU:")
                print(f"    image tensor device: {images[0].device} if images
else 'N/A'")

            images = [img.to(device) for img in images]
            targets = [{k: v.to(device) for k, v in t.items()} for t in
targets]

            if batch_idx == 0:
                print(f"    Тензоры ПОСЛЕ перемещения на GPU:")
                print(f"    image tensor device: {images[0].device}")
                if torch.cuda.is_available():
                    print(f"    VRAM занято после загрузки батча:
{torch.cuda.memory_allocated(0) / 1024**3:.2f} GB")

            loss_dict = model(images, targets)
            losses = sum(loss for loss in loss_dict.values())

```



```

        optimizer.zero_grad()
        losses.backward()
        optimizer.step()

        total_loss += losses.item()
        num_batches += 1
        progress_bar.set_postfix({'loss': losses.item()})

    except Exception as e:
        print(f"Ошибка в батче: {e}")
        import traceback
        traceback.print_exc()
        continue

    if torch.cuda.is_available():
        print(f"    VRAM занято после эпохи: {torch.cuda.memory_allocated(0)
/ 1024**3:.2f} GB")

    return total_loss / max(1, num_batches)

num_epochs = 10
train_losses = []

print("\n" + "="*50)
print("НАЧАЛО ОБУЧЕНИЯ")
print("="*50)

for epoch in range(1, num_epochs + 1):
    if len(train_loader) == 0:
        print(f"Ошибка: train_loader пуст на эпохе {epoch}")
        break

    if torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()
        print(f"\n Эпоха {epoch} - начало, VRAM:
{torch.cuda.memory_allocated(0) / 1024**3:.2f} GB")

    train_loss = train_one_epoch(model, optimizer, train_loader, device,
epoch)
    train_losses.append(train_loss)
    print(f"Epoch {epoch}/{num_epochs}, Loss: {train_loss:.4f}")
    print("-"*30)

if len(train_losses) > 0:
    print("\nОбучение завершено успешно!")

```

```

plt.figure(figsize=(10, 6))
plt.plot(range(1, len(train_losses) + 1), train_losses, 'b-',
linewidth=2, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')
plt.grid(True, alpha=0.3)
plt.show()
else:
    print("\nОбучение не выполнено из-за отсутствия данных")

```

Проверка перед созданием DataLoader:

```

Train dataset size: 600
Val dataset size: 0
GPU доступен: True
Устройство: cuda
GPU: NVIDIA GeForce RTX 4060 Laptop GPU
VRAM всего: 8.0 GB
VRAM занято до обучения: 2.49 GB

```

ВНИМАНИЕ: Валидационный датасет пуст!  
 DataLoader для обучения создан:  
 Количество батчей в train: 150

Модель на устройстве: cuda:0

```

=====
НАЧАЛО ОБУЧЕНИЯ
=====

```

Эпоха 1 - начало, VRAM: 2.49 GB

```

{"model_id": "00a8fd245e66479ebb03ec05415b96c6", "version_major": 2, "version_min
or": 0}

```

```

Тензоры ДО перемещения на GPU:
  image tensor device: cpu
Тензоры ПОСЛЕ перемещения на GPU: image
  tensor device: cuda:0
  VRAM занято после загрузки батча: 2.49 GB
  VRAM занято после эпохи: 2.80 GB
Epoch 1/10, Loss: 0.2239
-----

```

Эпоха 2 - начало, VRAM: 2.80 GB

```

{"model_id": "309863d8e7594f89bafcf0ba14068cacb", "version_major": 2, "version_min
or": 0}

```

Тензоры ДО перемещения на GPU: image  
tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
tensor device: cuda:0  
VRAM занято после загрузки батча: 2.80 GB  
VRAM занято после эпохи: 2.80 GB Epoch 2/10, Loss: 0.1167  
-----

Эпоха 3 - начало, VRAM: 2.80 GB

```
{"model_id":"3bcef0ac60684431a4c25bfa0f467c47","version_major":2,"version_min  
or":0}
```

Тензоры ДО перемещения на GPU:  
image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
tensor device: cuda:0  
VRAM занято после загрузки батча: 2.80 GB  
VRAM занято после эпохи: 2.80 GB  
Epoch 3/10, Loss: 0.1012  
-----

Эпоха 4 - начало, VRAM: 2.80 GB

```
{"model_id":"618b78f29d5544409c4b892094a8300c","version_major":2,"version_min  
or":0}
```

Тензоры ДО перемещения на GPU:  
image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
tensor device: cuda:0  
VRAM занято после загрузки батча: 2.80 GB  
VRAM занято после эпохи: 2.80 GB  
Epoch 4/10, Loss: 0.0912  
-----

Эпоха 5 - начало, VRAM: 2.80 GB

```
{"model_id":"f467229228bb4de7a12f0703c7c4e585","version_major":2,"version_min  
or":0}
```

Тензоры ДО перемещения на GPU:  
image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
tensor device: cuda:0  
VRAM занято после загрузки батча: 2.80 GB

VRAM занято после эпохи: 2.80 GB  
Epoch 5/10, Loss: 0.0903  
-----

Эпоха 6 - начало, VRAM: 2.80 GB

```
{"model_id":"fb29c5c962744e0c82d016450973be24","version_major":2,"version_min  
or":0}
```

Тензоры ДО перемещения на GPU:  
image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
tensor device: cuda:0  
VRAM занято после загрузки батча: 2.80 GB  
VRAM занято после эпохи: 2.80 GB  
Epoch 6/10, Loss: 0.0864  
-----

Эпоха 7 - начало, VRAM: 2.80 GB

```
{"model_id":"c1ccee221d1146598dbd0178172cbb7c","version_major":2,"version_min  
or":0}
```

Тензоры ДО перемещения на GPU:  
image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
tensor device: cuda:0  
VRAM занято после загрузки батча: 2.80 GB  
VRAM занято после эпохи: 2.80 GB  
Epoch 7/10, Loss: 0.0859  
-----

Эпоха 8 - начало, VRAM: 2.80 GB

```
{"model_id":"c20508c6b02a48eab76f59544c39bf29","version_major":2,"version_min  
or":0}
```

Тензоры ДО перемещения на GPU:  
image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
tensor device: cuda:0  
VRAM занято после загрузки батча: 2.80 GB  
VRAM занято после эпохи: 2.80 GB  
Epoch 8/10, Loss: 0.0867  
-----

Эпоха 9 - начало, VRAM: 2.80 GB

```
{"model_id":"50b9d04b9b214865ac9e75f9dff70d2a","version_major":2,"version_min  
or":0}
```

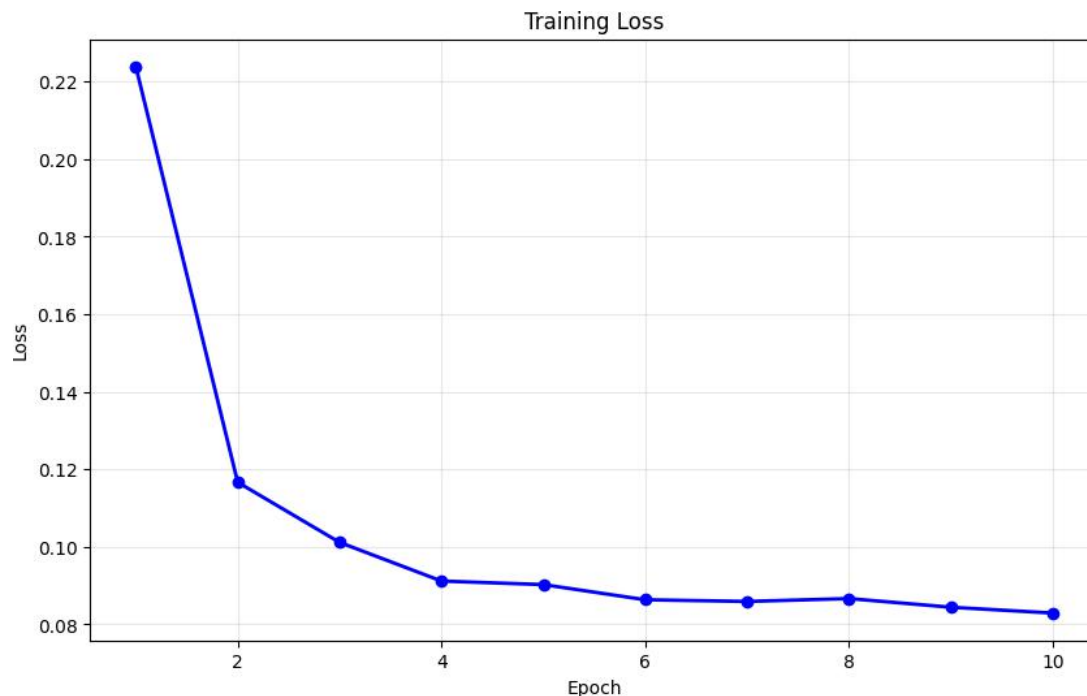
```
Тензоры ДО перемещения на GPU:  
  image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
  tensor device: cuda:0  
  VRAM занято после загрузки батча: 2.80 GB  
  VRAM занято после эпохи: 2.80 GB  
Epoch 9/10, Loss: 0.0844  
-----
```

Эпоха 10 - начало, VRAM: 2.80 GB

```
{"model_id":"ab1b55ed8478410babc11a5d2baa399e","version_major":2,"version_min  
or":0}
```

```
Тензоры ДО перемещения на GPU:  
  image tensor device: cpu  
Тензоры ПОСЛЕ перемещения на GPU: image  
  tensor device: cuda:0  
  VRAM занято после загрузки батча: 2.80 GB  
  VRAM занято после эпохи: 2.80 GB  
Epoch 10/10, Loss: 0.0829  
-----
```

Обучение завершено успешно!



**Детекция с поддержкой нескольких объектов** Определяются вспомогательные функции:

**is\_white\_background** – определяет, является ли фон преимущественно белым.

**remove\_white\_background** – заменяет белый фон на серый, выделяя контур объекта. **apply\_nms** – Non-Maximum Suppression для удаления пересекающихся bounding boxes.

**detect\_multiple\_objects** – полный пайплайн детекции: загрузка изображения, предобработка (удаление белого фона), нормализация, инференс модели, фильтрация по порогу уверенности, NMS, визуализация результатов на изображении.

Выбирается тестовое изображение (например, `peag.png`).

Выполняется детекция с низким порогом уверенности (0.15) для обнаружения всех возможных объектов.

Результаты выводятся: количество найденных объектов после фильтрации и NMS, визуализация с bounding boxes, лучший прогноз с процентом уверенности.

```
import cv2
import numpy as np
from PIL import Image, ImageDraw
```

```
def is_white_background(image, threshold=240, white_ratio=0.3):
```

```

img_np = np.array(image)

if img_np.shape[-1] == 4:
    img_np = img_np[:, :, :3]

white_pixels = np.all(img_np >= threshold, axis=2)
white_percentage = np.sum(white_pixels) / white_pixels.size

return white_percentage > white_ratio

def remove_white_background(image, threshold=240):

    img_np = np.array(image)

    if img_np.shape[-1] == 4:
        img_np = img_np[:, :, :3]

    white_mask = np.all(img_np >= threshold, axis=2)
    object_mask = ~white_mask

    h, w = img_np.shape[:2]

    contours, _ = cv2.findContours(object_mask.astype(np.uint8) * 255,
                                   cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)

    if contours:
        largest_contour = max(contours, key=cv2.contourArea)

        clean_mask = np.zeros((h, w), dtype=np.uint8)
        cv2.drawContours(clean_mask, [largest_contour], -1, 255, -1)

        kernel = np.ones((5, 5), np.uint8)
        clean_mask = cv2.dilate(clean_mask, kernel, iterations=2)

        result = np.full_like(img_np, 128)
        result[clean_mask == 255] = img_np[clean_mask == 255]

        return Image.fromarray(result)

    return image

def apply_nms(bboxes, scores, labels, iou_threshold=0.4):

    if len(bboxes) == 0:
        return bboxes, scores, labels

```

```

x1 = boxes[:, 0]
y1 = boxes[:, 1]
x2 = boxes[:, 2]
y2 = boxes[:, 3]

areas = (x2 - x1 + 1) * (y2 - y1 + 1)
order = scores.argsort()[::-1]

keep = []
while order.size > 0:
    i = order[0]
    keep.append(i)

    xx1 = np.maximum(x1[i], x1[order[1:]])
    yy1 = np.maximum(y1[i], y1[order[1:]])
    xx2 = np.minimum(x2[i], x2[order[1:]])
    yy2 = np.minimum(y2[i], y2[order[1:]])

    w = np.maximum(0.0, xx2 - xx1 + 1)
    h = np.maximum(0.0, yy2 - yy1 + 1)
    inter = w * h
    iou = inter / (areas[i] + areas[order[1:]] - inter)

    order = order[np.where(iou <= iou_threshold)[0] + 1]

return boxes[keep], scores[keep], labels[keep]

def detect_multiple_objects(image_path, model, device, val_dataset,
                           confidence_threshold=0.15,
                           use_nms=True,
                           iou_threshold=0.4,
                           max_detections=10):

    original = Image.open(image_path).convert("RGB")

    has_white_bg = is_white_background(original)

    print(f"\n{'='*50}")
    print(f" Анализ изображения: {image_path}")
    print(f" Фон: {'БЕЛЫЙ' if has_white_bg else 'НЕ БЕЛЫЙ'}")

    if has_white_bg:
        print(" Удаляем белый фон...")
        processed = remove_white_background(original)
        cropped = processed
    else:

```



```

    print("Фон не белый, оставляем без изменений")
    cropped = original

print(f"    Размер изображения: {cropped.size[0]}x{cropped.size[1]}")

transform =
    T.Compose([ T.Resize((224,
224)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

img_tensor = transform(cropped).unsqueeze(0).to(device)

model.eval()
with torch.no_grad():
    pred = model(img_tensor)[0]

boxes = pred['boxes'].cpu().numpy()
scores = pred['scores'].cpu().numpy()
labels = pred['labels'].cpu().numpy()

print(f"\n Результаты детекции:")
print(f"    Всего предсказаний: {len(scores)}")

mask = scores >= confidence_threshold
boxes, scores, labels = boxes[mask], scores[mask], labels[mask]
print(f"    После фильтрации (порог {confidence_threshold}):
{len(scores)}")

# Non-Maximum Suppression
if use_nms and len(boxes) > 0:
    boxes, scores, labels = apply_nms(boxes, scores, labels,
iou_threshold)
    print(f"    После NMS (IoU={iou_threshold}): {len(scores)}")

if len(scores) > max_detections:
    top_indices = np.argsort(scores)[-max_detections:]
    boxes = boxes[top_indices]
    scores = scores[top_indices]
    labels = labels[top_indices]
    print(f"    Ограничено до {max_detections} объектов")

fig, ax = plt.subplots(1, figsize=(14, 10))

```

```

ax.imshow(cropped)

colors = ['red', 'blue', 'green', 'orange', 'purple', 'cyan', 'magenta',
'yellow', 'brown', 'pink']

for i, (box, score, label) in enumerate(zip(boxes, scores, labels)):
    x1, y1, x2, y2 = box

    scale_x = cropped.size[0] / 224
    scale_y = cropped.size[1] / 224
    x1, x2 = x1 * scale_x, x2 * scale_x
    y1, y2 = y1 * scale_y, y2 * scale_y

    color = colors[i % len(colors)]

    rect = patches.Rectangle((x1, y1), x2-x1, y2-y1,
                             linewidth=2, edgecolor=color,
facecolor='none')
    ax.add_patch(rect)

    class_name = val_dataset.idx_to_class.get(int(label),
f"Class_{label}")
    ax.text(x1, y1-5, f'{i+1}. {class_name}: {score:.1%}', fontsize=10,
            color=color, weight='bold', bbox=dict(facecolor='white',
            alpha=0.8, edgecolor='none'))

ax.set_title(f"Обнаружено объектов: {len(boxes)}", fontsize=14)
ax.axis('off')
plt.tight_layout()
plt.show()

if len(scores) > 0:
    best_idx = np.argmax(scores)
    best_score = scores[best_idx]
    best_label = labels[best_idx]
    best_class = val_dataset.idx_to_class.get(int(best_label),
f"Class_{best_label}")

    print(f"\n ЛУЧШИЙ ПРОГНОЗ:")
    print(f"    Объект: {best_class}")
    print(f"    Уверенность: {best_score:.1%}")
else:
    print(f"\nОбъекты не обнаружены")

return boxes, scores, labels

# =====

```

```
# ИСПОЛЬЗОВАНИЕ
```

```
# =====
```

```
IMAGE_PATH = r"C:\Users\admin_local\Documents\AIR\AI_metod\pear.png"
```

```
boxes, scores, labels = detect_multiple_objects(  
    IMAGE_PATH,  
    model,  
    device,  
    val_dataset,  
    confidence_threshold=0.15,    # НИЗКИЙ порог для обнаружения всех объектов  
    use_nms=True,                # Включаем подавление дубликатов  
    iou_threshold=0.4,           # Порог пересечения боксов  
    max_detections=10            # Максимальное количество объектов  
)
```

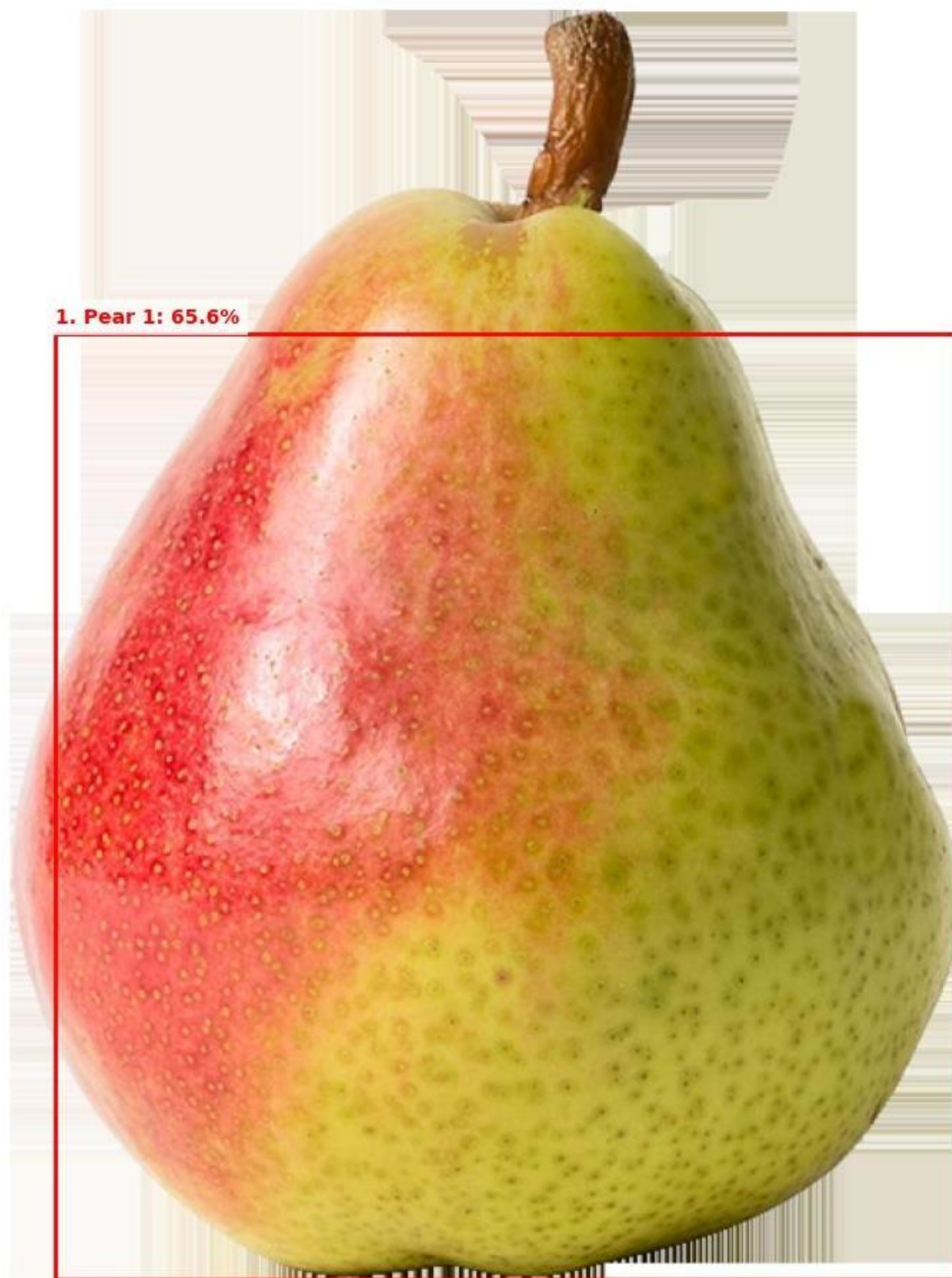
```
=====
```

```
Анализ изображения: C:\Users\admin_local\Documents\AIR\AI_metod\pear.png Фон:  
НЕ БЕЛЫЙ  
Фон не белый, оставляем без изменений  
Размер изображения: 509x689
```

```
Результаты детекции:
```

```
Всего предсказаний: 5  
После фильтрации (порог 0.15): 2  
После NMS (IoU=0.4): 1
```

Обнаружено объектов: 1



1. Pear 1: 65.6%

ЛУЧШИЙ ПРОГНОЗ:

Объект: Pear 1

Уверенность: 65.6%

## сохранение модели

```
model_save_path = "fruit_vegetable_detector.pth"
torch.save({
    'model_state_dict': model.state_dict(),
    'class_names': classes,
    'idx_to_class': val_dataset.idx_to_class,
    'num_classes': num_classes
}, model_save_path)

print(f"\nМодель сохранена: {model_save_path}")
print(f" Размер файла: {os.path.getsize(model_save_path) / 1024 / 1024:.2f} MB")

def load_trained_model(model_path, device):
    checkpoint = torch.load(model_path, map_location=device)
    model = create_model(checkpoint['num_classes'])
    model.load_state_dict(checkpoint['model_state_dict'])
    model.to(device)
    model.eval()
    return model, checkpoint['idx_to_class']
```

Модель сохранена: fruit\_vegetable\_detector.pth  
Размер файла: 158.16 MB